Lecture 9 — Advanced Graphs: Bridges, Articulation Points, Biconnected Components, 2-SAT, Euler Tours

> National Championship Prep · Prerequisites: Year weeks 16–17 (BFS/DFS), 20 (trees, diameter), 21 (SCC: Kosaraju/Tarjan), 22 (LCA); Nationals 04 (HLD) for the block-cut-tree applications · Estimated time: 2 × 75 min lecture + 3 h practice

1. Overview

This lecture collects four linear-time graph techniques that national-level sets use as building blocks:

- **Lowlink DFS** finds bridges, articulation (cut) vertices, and biconnected components in one O(n + m) pass.
- **Compression to trees.** Contracting 2-edge-connected components gives the **bridge tree**; blocks and cut vertices give the **block-cut tree**. Many "every path must use..." questions become path questions on a tree, solvable with diameter, LCA, or HLD.
- **2-SAT** decides satisfiability of a conjunction of 2-literal clauses in O(n + m) via SCCs of the implication graph, and produces an assignment.
- **Euler paths and circuits**: existence by degree conditions, construction by Hierholzer's algorithm in O(n + m).

The algorithms are short; the difficulty in contests is **modeling** (recognizing that the constraints are 2-SAT, that the walk is an Euler trail on a cleverly chosen graph) and **edge cases** (multi-edges, self-loops, disconnected input, recursion depth).

2. Lowlink: bridges and articulation points

2.1 Definitions

Run a DFS on an undirected graph and record the entry time `tin[v]`. In an undirected DFS every non-tree edge connects a vertex to one of its ancestors (a **back edge**) — there are no cross edges, because if u and v were in different branches, whichever was visited first would have explored the edge to the other. Define

```
low[v] = min( tin[v], tin[u] for every back edge (w, u) with w in subtree(v) )
```

computed bottom-up as `low[v] = min(tin[v], tin[u] over back edges v–u, low[c] over children c)`.

2.2 Criteria and proofs

Bridges. A tree edge (v, c) is a bridge iff `low[c] > tin[v]`.

Proof. If `low[c] > tin[v]`, then every edge with one endpoint in subtree(c) and the other outside is either (v, c) or a back edge to an ancestor with `tin ≤ tin[v]` — impossible by assumption. So deleting (v, c) disconnects subtree(c). Conversely, if `low[c] ≤ tin[v]`, there is a back edge from some w in subtree(c) to an ancestor u of c with `tin[u] ≤ tin[v]`; the tree path u ⇝ v → c ⇝ w plus the back edge w–u is a cycle through (v, c), so it is not a bridge. Back edges always lie on such a cycle, so they are never bridges. ∎

Cut vertices. A non-root v is a cut vertex iff some child c has `low[c] ≥ tin[v]`; the root is a cut vertex iff it has at least two DFS children.

Proof. For a non-root v, deleting v separates subtree(c) from v's parent unless some back edge from subtree(c) reaches a *proper* ancestor of v, i.e. unless `low[c] < tin[v]`. Different children of v are never connected by edges (no cross edges), so the condition per child is exact. For the root, its children's subtrees are pairwise disconnected without the root, so it separates the graph iff there are at least two. ∎

2.3 Multi-edges and self-loops

The DFS must skip the **edge** it arrived by, not every edge to the parent vertex. Two parallel edges between u and v form a cycle, so neither is a bridge — skipping by vertex would report one. Store edge ids in the adjacency list and skip `id == parentEdge`. Self-loops are irrelevant to bridges and cut vertices; the reference code silently ignores them (they are not assigned to any block).

2.4 Recursion depth

A path graph with $n = 10^6$ makes the DFS 10^6 frames deep. Many judges run with an 8 MB or 64 MB stack, which overflows long before that. Options: an iterative DFS that stores `(vertex, parentEdge, next adjacency index)` on an explicit stack, or (when allowed) increasing the stack size. Know which your judge permits before the contest — Lecture 1's notebook checklist.

3. Biconnected components

3.1 2-edge-connected components and the bridge tree

Delete all bridges; the remaining connected components are the **2-edge-connected components**: two vertices are in the same component iff there are two edge-disjoint paths between them (Menger). Contracting each component gives a forest whose edges are exactly the bridges — the **bridge tree**.

- CF 1000E *We Need More Bosses*: the edges that separate s from t are the bridges on the bridge-tree path between their components; maximize ⇒ the tree diameter.
- Robbins' theorem: a connected graph has an orientation making it strongly connected iff it has no bridges. Orient DFS tree edges downward and back edges upward. Every vertex can climb to the root: from v, go down into its subtree to the vertex whose back edge achieves `low`, jump up, and repeat; bridgelessness guarantees each jump strictly decreases the reachable `tin` until the root. (CSES 2177, CF 118E.)
- CF 732F *Tourist Reform*: orient the largest 2-edge-connected component strongly and point every bridge toward it; the minimum reachability equals that component's size.

3.2 Blocks and the block-cut tree

A **block** (biconnected component) is a maximal subgraph without a cut vertex of its own: either a single bridge or a maximal 2-vertex-connected subgraph. Blocks partition the **edges**; a cut vertex belongs to several blocks.

To compute blocks, push each tree edge and each back edge (once, from the lower endpoint) onto an edge stack. When the DFS returns from child c to v with `low[c] ≥ tin[v]`, the edges above and including (v, c) on the stack form exactly one block: they are the edges of subtree(c) plus back edges into v, minus blocks already popped deeper. Pop them.

The **block-cut tree** has a node for each block and for each cut vertex, with an edge between a block and each cut vertex it contains. It is a tree (a forest if the graph is disconnected). Key property: the vertices that lie on **every** simple u–v path are exactly the cut vertices on the block-cut-tree path between u and v, and the set of vertices on **some** simple u–v path is the union of the blocks on that tree path.

- CF 487E *Tourists* (stretch): each vertex has a price; queries ask the minimum price over all vertices lying on some simple u–v path, with price updates. Build the block-cut tree in the variant with a node for **every** vertex (not only cut vertices) plus a node per block, store at each block node the min over its non-parent vertices (a multiset), and answer path minima with HLD (Lecture 4).
- CF 962F *Simple Cycles Edges*: an edge lies on exactly one simple cycle iff its block is a simple cycle, i.e. the block has as many edges as vertices.

4. 2-SAT

4.1 Implication graph

Variables x_0 ... x_{n−1}; literal `2x` means "x is true", `2x+1` means "x is false", so negation is `^ 1`. A clause (a ∨ b) is equivalent to the two implications ¬a → b and ¬b → a; add both edges. The graph has 2n vertices and is **skew-symmetric**: u → v is an edge iff ¬v → ¬u is.

Theorem. The formula is satisfiable iff no variable x has x and ¬x in the same strongly connected component.

Necessity: if x ⇝ ¬x and ¬x ⇝ x, any assignment makes some implication on these paths fail. *Sufficiency* is by construction: order the SCCs topologically (edges go forward) and set x true iff the SCC of x comes after the SCC of ¬x. Suppose clause (a ∨ b) were violated, so a and b are both false: then a ≺ ¬a and b ≺ ¬b in the order; the edge ¬a → b gives ¬a ≤ b and the edge ¬b → a gives ¬b ≤ a; chaining, a ≺ ¬a ≤ b ≺ ¬b ≤ a, a contradiction.

Tarjan's algorithm finishes SCCs in **reverse** topological order, so with Tarjan ids the rule is `value[x] = id[x] < id[¬x]`. With Kosaraju (components numbered in topological order of the first pass) it is the opposite. This inversion is the single most common 2-SAT bug.

4.2 Modeling

- Forcing: `a` must hold ⇒ clause (a ∨ a), i.e. edge ¬a → a.
- Implication a → b ⇒ (¬a ∨ b). Equality ⇒ two implications each way. Xor ⇒ (a ∨ b) ∧ (¬a ∨ ¬b).
- **At most one** of a_1…a_k without $O(k^2)$ clauses: introduce prefix variables p_i ("some of a_1..a_i is true") with $a_i \to p_i$, p_{i−1} → p_i, and p_{i−1} → $\neg a_i$. This is how CF 1215F *Radio Stations* handles an ordered range of frequencies (variables "frequency ≥ t" with chain implications).
- Recognizing 2-SAT: every entity has **two states**, and every constraint mentions **at most two entities**. CF 776D *The Door Problem* (each door is controlled by exactly two switches, so "door ends open" is an equality or xor between two switch variables); CF 228E (each road toggled by its endpoints' choices); CSES 1684 *Giant Pizza* (each member's two wishes form a clause).
- Binary search + 2-SAT: "maximize the minimum distance between chosen points where each item picks one of two positions" — for a candidate distance, conflicting pairs give clauses.

5. Euler paths and circuits

5.1 Conditions

An **Euler trail** uses every edge exactly once. Ignoring isolated vertices, all edges must be in one connected component (weakly connected for directed graphs). Then:

- Undirected: a circuit exists iff every degree is even; a trail exists iff exactly 0 or 2 vertices have odd degree (start at one of them).
- Directed: a circuit exists iff in = out at every vertex; a trail exists iff at most one vertex has out − in = 1 (the start), at most one has in − out = 1 (the end), and all others are balanced.

Necessity: each pass through a vertex uses one edge in and one edge out. Sufficiency follows from Hierholzer's construction.

5.2 Hierholzer's algorithm

Start at the correct vertex and keep a stack. Repeatedly look at the top v: if it has an unused edge, mark it used and push the other endpoint; otherwise pop v and append it to the answer. The answer, reversed, is the trail.

Why it works: the walk from the start can only get stuck at the end vertex of the trail (degree parity), at which point that vertex is emitted; backing up, any vertex with remaining edges starts a closed sub-tour that is spliced in at that point. Each edge is traversed once; a per-vertex pointer into the adjacency list ensures each adjacency entry is examined O(1) times, so the algorithm is O(n + m). If the emitted path has fewer than m + 1 vertices, some edges were unreachable — the graph was not connected.

5.3 Modeling with Euler tours

- **De Bruijn sequences** (CSES 1692): for binary strings of length k, vertices are (k−1)-bit words and each k-bit word is an edge from its prefix to its suffix. Every vertex has in = out = 2, so an Euler circuit exists; it spells a shortest string containing every k-bit word.
- **Reconstruct a string from its length-3 substrings** (CF 508D *Tanya and Password*): vertices are 2-letter pairs, substrings are edges; an Euler trail gives the password.
- **Balanced orientation** (CF 723E *One-Way Reform*): connect every odd-degree vertex to a new dummy vertex (there is an even number of them), find Euler circuits, and orient edges along them. Every original even-degree vertex gets in = out, which is optimal.
- **Chinese postman** on undirected graphs: duplicate edges along a minimum-weight perfect matching of odd vertices (via shortest-path distances), then take an Euler circuit.

6. Reference implementations

All programs compile with `g++ -std=c++17 -O2 -Wall` and were stress-tested on hundreds of random small graphs: bridges and cut vertices against "remove it and count components", blocks against the definition "two edges are in the same block iff they lie on a common simple cycle" (all simple cycles enumerated, multi-edges included), 2-edge components against "no single edge separates u and v", 2-SAT against exhaustive assignments (and the produced assignment is verified), Euler trails against brute force over edge orderings (and the produced trail is verified).

6.1 Bridges, articulation points, blocks, 2-edge-connected components

Input: n m, then m edges `u v` (0-indexed, multi-edges allowed). Output: bridges (edge ids), cut vertices, the blocks as sorted edge-id lists, and a 2-edge-component label per vertex.

```
#include <bits/stdc++.h>
using namespace std;

// Lowlink DFS on an undirected multigraph (vertices 0..n-1, edges by id).
// Finds bridges, articulation points, vertex-biconnected components (as edge
// lists) and 2-edge-connected components (as vertex labels).
struct Lowlink {
    int n, timer = 0;
    vector<vector<pair<int,int>>> adj;        // (neighbor, edge id)
    vector<pair<int,int>> edges;
    vector<int> tin, low, edgeStack;
    vector<bool> isBridge, isCut;
    vector<vector<int>> bccs;                 // each = list of edge ids

    Lowlink(int n) : n(n), adj(n) {}
    void addEdge(int u, int v) {
        adj[u].push_back({v, (int)edges.size()});
        adj[v].push_back({u, (int)edges.size()});
        edges.push_back({u, v});
    }
    void dfs(int v, int parentEdge) {
        tin[v] = low[v] = timer++;
        int children = 0;
        for (auto [to, id] : adj[v]) {
            if (id == parentEdge) continue;              // skip only this edge, not all
to parent
            if (tin[to] != -1) {                          // back edge (or forward
duplicate)
                if (tin[to] < tin[v]) edgeStack.push_back(id);
                low[v] = min(low[v], tin[to]);
                continue;
            }
            edgeStack.push_back(id);
            dfs(to, id);
            children++;
```

```
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v]) isBridge[id] = true;
            if (low[to] >= tin[v]) {                      // v separates to's subtree
                if (parentEdge != -1) isCut[v] = true;
                vector<int> comp;
                while (true) {
                    int e = edgeStack.back(); edgeStack.pop_back();
                    comp.push_back(e);
                    if (e == id) break;
                }
                bccs.push_back(comp);
            }
        }
        if (parentEdge == -1 && children >= 2) isCut[v] = true;
    }
    void run() {
        tin.assign(n, -1); low.assign(n, 0);
        isBridge.assign(edges.size(), false); isCut.assign(n, false);
        for (int v = 0; v < n; v++) if (tin[v] == -1) dfs(v, -1);
    }
    // 2-edge-connected components: connected components after deleting bridges.
    vector<int> twoEdgeComponents() const {
        vector<int> comp(n, -1);
        int c = 0;
        for (int s = 0; s < n; s++) {
            if (comp[s] != -1) continue;
            vector<int> st = {s}; comp[s] = c;
            while (!st.empty()) {
                int v = st.back(); st.pop_back();
                for (auto [to, id] : adj[v])
                    if (!isBridge[id] && comp[to] == -1) { comp[to] = c;
st.push_back(to); }
            }
            c++;
        }
        return comp;
    }
};

int main() {
    int n, m;
    cin >> n >> m;
    Lowlink g(n);
    for (int i = 0; i < m; i++) { int u, v; cin >> u >> v; g.addEdge(u, v); }
    g.run();
    vector<int> bridges, cuts;
    for (int i = 0; i < m; i++) if (g.isBridge[i]) bridges.push_back(i);
    for (int v = 0; v < n; v++) if (g.isCut[v]) cuts.push_back(v);
    cout << bridges.size() << ":"; for (int e : bridges) cout << " " << e; cout << "\n";
    cout << cuts.size() << ":"; for (int v : cuts) cout << " " << v; cout << "\n";
    // print each block as a sorted edge list, blocks sorted
    vector<vector<int>> b = g.bccs;
    for (auto& x : b) sort(x.begin(), x.end());
    sort(b.begin(), b.end());
    cout << b.size() << "\n";
```

```
    for (auto& x : b) { for (size_t i = 0; i < x.size(); i++) cout << x[i] << " \n"[i + 1
== x.size()]; }
    vector<int> c = g.twoEdgeComponents();
    for (int v = 0; v < n; v++) cout << c[v] << " \n"[v + 1 == n];
}
```

6.2 2-SAT

Input: n m, then m clauses `a b` with literals written as `+i` / `-i` (1-indexed). Output: `IMPOSSIBLE` or a sign per variable.

```
#include <bits/stdc++.h>
using namespace std;

// 2-SAT with Tarjan's SCC. Variable x has literals 2x (true) and 2x+1 (false).
struct TwoSat {
    int n;
    vector<vector<int>> g;
    vector<int> comp, tin, low, stk;
    vector<bool> onStack, value;
    int timer = 0, ncomp = 0;

    TwoSat(int n) : n(n), g(2 * n) {}
    static int lit(int x, bool isTrue) { return 2 * x + (isTrue ? 0 : 1); }
    // clause (a OR b), literals given as lit(...)
    void addOr(int a, int b) { g[a ^ 1].push_back(b); g[b ^ 1].push_back(a); }
    void addImplies(int a, int b) { addOr(a ^ 1, b); }
    void forceTrue(int a) { addOr(a, a); }

    void tarjan(int v) {
        tin[v] = low[v] = timer++;
        stk.push_back(v); onStack[v] = true;
        for (int to : g[v]) {
            if (tin[to] == -1) { tarjan(to); low[v] = min(low[v], low[to]); }
            else if (onStack[to]) low[v] = min(low[v], tin[to]);
        }
        if (low[v] == tin[v]) {
            while (true) {
                int u = stk.back(); stk.pop_back(); onStack[u] = false;
                comp[u] = ncomp;
                if (u == v) break;
            }
            ncomp++;                        // Tarjan numbers SCCs in reverse topological
order
        }
    }
    bool solve() {
        comp.assign(2 * n, -1); tin.assign(2 * n, -1); low.assign(2 * n, 0);
        onStack.assign(2 * n, false);
        for (int v = 0; v < 2 * n; v++) if (tin[v] == -1) tarjan(v);
        value.assign(n, false);
        for (int x = 0; x < n; x++) {
```

```
            if (comp[2 * x] == comp[2 * x + 1]) return false;
            // pick the literal whose SCC comes later in topological order (smaller
Tarjan id)
            value[x] = comp[2 * x] < comp[2 * x + 1];
        }
        return true;
    }
};

int main() {
    // Input: n m, then m clauses "a b" with literals as +i / -i (1-indexed), like
DIMACS.
    int n, m;
    cin >> n >> m;
    TwoSat ts(n);
    auto conv = [&](int v) { return TwoSat::lit(abs(v) - 1, v > 0); };
    for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; ts.addOr(conv(a), conv(b)); }
    if (!ts.solve()) { cout << "IMPOSSIBLE\n"; return 0; }
    for (int x = 0; x < n; x++) cout << (ts.value[x] ? '+' : '-') << " \n"[x + 1 == n];
}
```

6.3 Euler path / circuit (directed or undirected)

Input: `n m d` (d = 1 for directed), then m edges `u v` (0-indexed). Output: `IMPOSSIBLE` or the vertex sequence of an Euler trail. For CSES 1691 (circuit from vertex 1) call `eulerTrail` with the required start after checking that all degrees are even; for CSES 1693 check that the start is vertex 1 and the end is vertex n.

```
#include <bits/stdc++.h>
using namespace std;

// Hierholzer's algorithm (iterative). Returns the sequence of vertices of an
// Euler trail starting at `start` that uses every edge exactly once, or {} if
// the edges reachable from start are not all used (graph not connected enough).
// Degree conditions must be checked by the caller.
vector<int> eulerTrail(int n, const vector<pair<int,int>>& edges, bool directed, int
start) {
    int m = edges.size();
    vector<vector<pair<int,int>>> adj(n);          // (to, edge id)
    for (int i = 0; i < m; i++) {
        auto [u, v] = edges[i];
        adj[u].push_back({v, i});
        if (!directed) adj[v].push_back({u, i});
    }
    vector<int> ptr(n, 0), path, st = {start};
    vector<bool> used(m, false);
    while (!st.empty()) {
        int v = st.back();
        while (ptr[v] < (int)adj[v].size() && used[adj[v][ptr[v]].second]) ptr[v]++;
        if (ptr[v] == (int)adj[v].size()) { path.push_back(v); st.pop_back(); continue; }
        auto [to, id] = adj[v][ptr[v]++];
        used[id] = true;
```

```
        st.push_back(to);
    }
    if ((int)path.size() != m + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}

// Chooses a valid start vertex from the degree conditions; returns {} if none.
vector<int> eulerPath(int n, const vector<pair<int,int>>& edges, bool directed) {
    vector<int> out(n, 0), in(n, 0);
    for (auto [u, v] : edges) { out[u]++; in[v]++; }
    int start = -1;
    for (int v = 0; v < n; v++) if (out[v] + in[v] > 0) { start = v; break; }
    if (start == -1) return {0};                    // no edges: trivial trail
    if (directed) {
        int plus = 0, minus = 0;
        for (int v = 0; v < n; v++) {
            if (out[v] - in[v] == 1) { plus++; start = v; }
            else if (in[v] - out[v] == 1) minus++;
            else if (in[v] != out[v]) return {};
        }
        if (!((plus == 0 && minus == 0) || (plus == 1 && minus == 1))) return {};
    } else {
        int odd = 0;
        for (int v = 0; v < n; v++) if ((out[v] + in[v]) % 2) { odd++; start = v; }
        if (odd != 0 && odd != 2) return {};
    }
    return eulerTrail(n, edges, directed, start);
}

int main() {
    int n, m, d;
    cin >> n >> m >> d;                             // d = 1 for directed
    vector<pair<int,int>> edges(m);
    for (auto& [u, v] : edges) cin >> u >> v;
    vector<int> p = eulerPath(n, edges, d == 1);
    if (p.empty()) { cout << "IMPOSSIBLE\n"; return 0; }
    for (size_t i = 0; i < p.size(); i++) cout << p[i] << " \n"[i + 1 == p.size()];
}
```

7. Common mistakes

- **Skipping the parent vertex instead of the parent edge.** Breaks on multi-edges: a doubled edge is reported as a bridge.
- **Root rule for cut vertices.** The root is a cut vertex only with ≥ 2 DFS children; applying the `low[c] ≥ tin[v]` rule to the root marks every root as a cut vertex.
- **Using `low[to]` instead of `tin[to]` for back edges.** It happens to give correct bridges, but it breaks the articulation-point criterion (a back edge to v itself would inherit v's smaller `low`, hiding that v is a cut vertex). Use `tin[to]`.

- **Directed SCC vs undirected lowlink.** In Tarjan's SCC algorithm you must ignore edges to vertices that are finished and not on the stack; in the undirected lowlink there is no such check. Do not copy one into the other.
- **2-SAT: one implication per clause.** Every clause needs both ¬a → b and ¬b → a.
- **2-SAT: inverted assignment.** Tarjan ids are reverse topological; Kosaraju's (second pass on the transpose) are topological. Test on a tiny forced instance such as the single clause (x ∨ x).
- **Euler: no connectivity check**, wrong start vertex (directed: the vertex with out − in = 1), or marking only one of the two adjacency entries of an undirected edge as used (use edge ids).
- **Deep recursion** on path-like graphs — the classic national-final runtime error.

8. Practice problems

Bridges, cut vertices, biconnectivity:

- CSES 2076 — Necessary Roads — https://cses.fi/problemset/task/2076
- CSES 2077 — Necessary Cities — https://cses.fi/problemset/task/2077
- CSES 2177 — Strongly Connected Edges — https://cses.fi/problemset/task/2177
- Codeforces 118E — Bertown roads — https://codeforces.com/problemset/problem/118/E
- Codeforces 1000E — We Need More Bosses — https://codeforces.com/problemset/problem/1000/E
- Codeforces 732F — Tourist Reform — https://codeforces.com/problemset/problem/732/F
- Codeforces 962F — Simple Cycles Edges — https://codeforces.com/problemset/problem/962/F
- Codeforces 487E — Tourists (stretch) — https://codeforces.com/problemset/problem/487/E
- Library Checker — Two-Edge-Connected Components — https://judge.yosupo.jp/problem/two_edge_connected_components
- Library Checker — Biconnected Components — https://judge.yosupo.jp/problem/biconnected_components

2-SAT:

- CSES 1684 — Giant Pizza — https://cses.fi/problemset/task/1684
- Codeforces 776D — The Door Problem — https://codeforces.com/problemset/problem/776/D
- Codeforces 228E — The Road to Berland is Paved With Good Intentions — https://codeforces.com/problemset/problem/228/E
- Codeforces 1215F — Radio Stations — https://codeforces.com/problemset/problem/1215/F
- Library Checker — 2-SAT — https://judge.yosupo.jp/problem/two_sat

Euler tours:

- CSES 1691 — Mail Delivery — https://cses.fi/problemset/task/1691
- CSES 1693 — Teleporters Path — https://cses.fi/problemset/task/1693

- CSES 1692 — De Bruijn Sequence — https://cses.fi/problemset/task/1692
- Codeforces 508D — Tanya and Password — https://codeforces.com/problemset/problem/508/D
- Codeforces 723E — One-Way Reform — https://codeforces.com/problemset/problem/723/E
- Kattis — Eulerian Path — https://open.kattis.com/problems/eulerianpath

9. Further reading

- R. E. Tarjan, "Depth-first search and linear graph algorithms", SIAM J. Computing, 1972 — the original lowlink paper.
- J. Hopcroft, R. Tarjan, "Algorithm 447: efficient algorithms for graph manipulation", CACM, 1973 — biconnected components.
- B. Aspvall, M. Plass, R. Tarjan, "A linear-time algorithm for testing the truth of certain quantified boolean formulas", IPL, 1979 — 2-SAT via SCC.
- H. E. Robbins, "A theorem on graphs, with an application to a problem of traffic control", American Mathematical Monthly, 1939.
- cp-algorithms: "Finding bridges", "Finding articulation points", "2-SAT", "Euler path" — https://cp-algorithms.com/
- World Finals track, Lecture 4 (link-cut trees, virtual trees) and Lecture 5 (offline dynamic connectivity — bridges under edge insertions/deletions).